

TP2 – Réorganisons les graphes

L'objectif de ce TP est d'utiliser des interfaces, de l'héritage et généricité afin de d'illustrer les intérêts du polymorphisme, de la réutilisabilité et le découplage du code.

Contexte et motivation

Dans le premier TP, vous avez implémenté un graphe et un algorithme de plus court chemin. Cette conception fonctionne, mais elle est **spécifique** : le code est directement lié à une représentation unique des sommets et des arêtes.

Or, dans la pratique, les graphes peuvent varier considérablement :

- orientés ou non orientés,
- pondérés ou non,
- avec des sommets enrichis (coordonnées, labels, types),
- ou encore des sous-graphes et vues filtrées selon certaines propriétés.

Si le code reste couplé à une seule structure ou à un seul algorithme, toute adaptation à un nouveau type de graphe ou à un nouvel algorithme nécessite de **réécrire l'algorithme ou les classes existantes**, ce qui nuit à la réutilisabilité.

L'abstraction via des interfaces et la généricité permettent de faciliter la réutilisabilité de votre code :

- L'algorithme ne dépend que du **contrat minimal** du graphe (voisins et poids des arcs).
- Différentes représentations de graphes peuvent être introduites **sans modifier l'algorithme**.
- Plusieurs algorithmes de plus court chemin (Dijkstra, Bellman-Ford, A*, etc.) peuvent être implémentés et interchangeés facilement grâce au polymorphisme.
- On peut travailler avec des vues filtrées, des graphes dynamiques ou des sommets enrichis.

Étape 1 : Initialisation du projet à partir du tp précédent

1. Dans un terminal créez une copie de votre précédent projet:

```
cp -r tp1-graphs tp2-generic-graphs
```

2. Éditez et modifier le fichier pom.xml pour que l'artifactId soit tp2-generic-graphs
3. Nettoyez le projet avec mvn clean

Étape 2 : Une interface minimale pour les graphes

Analysez votre implémentation de l'algorithme du plus court chemin (classe `DijkstraSolver`) et identifiez les méthodes de la classe `Graph` qui sont nécessaires pour l'algorithme.

Réorganisez votre code pour faire apparaître une interface `Graph` définissant l'interface minimale requise par `DijkstraSolver` et garder l'implémentation de votre graphe orienté dans une classe `DirectedGraph`.

Proposez une implémentation de `UndirectedGraph` tout en minimisant le code à dupliquer.

Pensez à modifier vos tests et vérifier que tout fonctionne correctement.

Étape 3 : Un graphe générique

Identifiez l'interface minimale d'un sommet dont a besoin l'algorithme et votre modélisation d'un graphe. Quel contrat doivent respecter les sommets pour que votre algorithme fonctionne correctement ? Dans quelle classe de base de java ce contrat est-il décrit ?

A-t-on réellement besoin d'un type `Vertex` ? (la réponse est non, mais vous devriez savoir pourquoi).

Réorganisez votre code pour rendre vos classes génériques et non dépendantes du type `Vertex`.

Pensez à documenter votre classe `DijkstraSolver` pour rappeler le contrat (identifié précédemment) que doivent respecter les objets représentant les sommets.

Modifiez vos tests et vérifiez que tout fonctionne correctement.

Etape 4 : Robustesse et exceptions

Votre implémentation de graphe fonctionne elle si vous ajoutez des sommets ou des arêtes `null` ? Comment se comporte votre algorithme du plus court chemin si on passe des sommets n'appartenant pas au graphe ou des référence vers `null` ? Si ce n'est pas le cas, modifiez votre code pour lever (au plus tôt) des exceptions adaptées et ajoutez les tests correspondants.